

45 DAY FAANG INTERVIEW PREPARATION PLAN

DSA + BEHAVIORAL + LEADERSHIP + LLD/HLD

PAGE 1: DSA PLAN (45 DAYS)

Day	Dates	DSA Topic / Pattern	NeetCode Problems (3 Problems/Day)	✓
1	Day 1 (Jul 1)	Arrays & Hashing I	NC 1 - 3	☐
2	Day 2 (Jul 2)	Arrays & Hashing II	NC 4 - 6	☐
3	Day 3 (Jul 3)	Arrays & Hashing III	NC 7 - 9	☐
4	Day 4 (Jul 4)	Two Pointers I	NC 10 - 12	☐
5	Day 5 (Jul 5)	Two Pointers II	NC 13 - 15	☐
6	Day 6 (Jul 6)	Revision + Easy	Revise + Easy	☐
7	Day 7 (Jul 7)	Sliding Window I	NC 16 - 18	☐
8	Day 8 (Jul 8)	Sliding Window II	NC 19 - 21	☐
9	Day 9 (Jul 9)	Sliding Window III	NC 22 - 24	☐
10	Day 10 (Jul 10)	Stack I	NC 25 - 27	☐
11	Day 11 (Jul 11)	Stack II	NC 28 - 30	☐
12	Day 12 (Jul 12)	Monotonic Stack	NC 31 - 33	☐
13	Day 13 (Jul 13)	Queue / Deque	NC 34 - 36	☐
14	Day 14 (Jul 14)	Binary Search I	NC 37 - 39	☐
15	Day 15 (Jul 15)	Binary Search II	NC 40 - 42	☐

Day	Dates	DSA Topic / Pattern	NeetCode Problems	✓
16	Day 16 (Jul 16)	Linked List I	NC 43 - 45	☐
17	Day 17 (Jul 17)	Linked List II	NC 46 - 48	☐
18	Day 18 (Jul 18)	Linked List III	NC 49 - 51	☐
19	Day 19 (Jul 19)	Trees I (Basics)	NC 52 - 54	☐
20	Day 20 (Jul 20)	Trees II (BST)	NC 55 - 57	☐
21	Day 21 (Jul 21)	Trees III (Traversal)	NC 58 - 60	☐
22	Day 22 (Jul 22)	Trees IV (Advanced)	NC 61 - 63	☐
23	Day 23 (Jul 23)	BFS I	NC 64 - 66	☐
24	Day 24 (Jul 24)	BFS II	NC 67 - 69	☐
25	Day 25 (Jul 25)	DFS I	NC 70 - 72	☐
26	Day 26 (Jul 26)	DFS II	NC 73 - 75	☐
27	Day 27 (Jul 27)	Graphs (All)	NC 76 - 78	☐
28	Day 28 (Jul 28)	Heap / Priority Queue	NC 79 - 81	☐
29	Day 29 (Jul 29)	Backtracking I	NC 82 - 84	☐
30	Day 30 (Jul 30)	Backtracking II	NC 85 - 87	☐

PAGE 2: DSA PLAN (CONTINUE)

Day	Dates	DSA Topic / Pattern	NeetCode Problems	✓
31	Day 31 (Jul 31)	Backtracking III	NC 88 - 90	☐
32	Day 32 (Aug 1)	Intervals I	NC 91 - 93	☐
33	Day 33 (Aug 2)	Intervals II	NC 94 - 96	☐
34	Day 34 (Aug 3)	Greedy I	NC 97 - 99	☐
35	Day 35 (Aug 4)	Greedy II	NC 100 - 102	☐
36	Day 36 (Aug 5)	Dynamic Programming I	NC 103 - 105	☐
37	Day 37 (Aug 6)	Dynamic Programming II	NC 106 - 108	☐
38	Day 38 (Aug 7)	Dynamic Programming III	NC 109 - 111	☐
39	Day 39 (Aug 8)	DP (Advanced)	NC 112 - 114	☐
40	Day 40 (Aug 9)	Mixed Practice I	NC 115 - 117	☐
41	Day 41 (Aug 10)	Mixed Practice II	NC 118 - 120	☐
42	Day 42 (Aug 11)	Mixed Practice III	NC 121 - 123	☐
43	Day 43 (Aug 12)	Contest / Hard I	NC 124 - 126	☐
44	Day 44 (Aug 13)	Contest / Hard II	NC 127 - 129	☐
45	Day 45 (Aug 14)	Final Revision & Mock	All Revise	☐

DAILY STUDY TIMETABLE (MON - FRI)

8:30 - 10:30 PM	Solve 2 Problems (P1 + P2)
10:30 - 11:00 PM	Dry Run + Complexity + Explain
12:00 - 12:45 AM	Solve 3 rd Problem (P3)
12:45 - 1:00 AM	Behavioral Topic Practice
1:00 - 1:15 AM	Leadership / Communication Practice
1:15 - 1:30 AM	LLD / HLD Concept

WEEKEND (SAT - SUN)

- ✓ Solve 1-2 Easy/Medium Problems
- ✓ Revise notes, patterns, templates
- ✓ Mock Interview / Review
- ✓ System Design Deep Dive / Case Study

GOAL → CONSISTENCY > MOTIVATION
SHOW UP EVERYDAY. 45 DAYS CAN CHANGE YOUR CAREER!

PAGE 3: BEHAVIORAL + LEADERSHIP + LLD/HLD PLAN

Day	Behavioral Topic (1:00 - 1:40 AM slot)	Leadership / Communication (1:40 - 2:00 AM slot)	LLD / HLD Topic (15 min)
1	Introduction	Sr Director 1:1	LLD: SOLID
2	Achievement	Weekly Update	HLD: URL Shortener
3	Failure	Exec Summary	LLD: Factory
4	Conflict	Status Mail	HLD: Rate Limiter
5	Ownership	Design Review	LLD: Strategy
6	Leadership	Sprint Planning	HLD: Notification
7	Mentoring	Roadmap	LLD: Observer
8	Stakeholder	Stakeholder Update	HLD: Chat System
9	Influence	Influence Without Authority	LLD: Singleton
10	Ambiguity	Drive Meeting	HLD: News Feed
11	Customer Focus	Customer Discussion	LLD: Decorator
12	Deadline	Demo / PoC	HLD: File Storage
13	Innovation	ADR (Architecture Decision)	LLD: Builder
14	Decision Making	Escalation Handling	HLD: Search Engine
15	Learning Agility	Prioritization	LLD: Adapter

16	Disagreement	Delegation	HLD: E-commerce
17	Feedback	Decision Making	LLD: Facade
18	Teamwork	Conflict Resolution	HLD: Payment System
19	Pressure	Alignment	LLD: Proxy
20	Risk Taking	Risk Identification	HLD: Ride Sharing
21	Prioritization	Incident Review	LLD: Command
22	Delivery	RCA (Root Cause Analysis)	HLD: Rate Limiter 2.0
23	Incident Handling	Architecture Review	LLD: Composite
24	Debugging	Executive Mail	HLD: Social Network
25	Architecture	Customer Meeting	LLD: Flyweight
26	Trade-offs	Product Discussion	HLD: Ticket Booking
27	Quality	Technical Debt	LLD: Bridge
28	Ownership 2	Cross Team Sync	HLD: Video Streaming
29	Vision	Promotion Talk	LLD: Template Method
30	Influence 2	Mentoring / Coaching	HLD: Content Delivery

31	Promotion	Hiring Discussion	LLD: State
32	Manager Relationship	Feedback Delivery	HLD: Distributed Cache
33	Director Discussion	Budget Planning	LLD: Chain of Resp.
34	Executive Communication	Stakeholder Mgmt.	HLD: Leaderboard
35	Negotiation	Negotiation	LLD: Mediator
36	Hiring & Onboarding	Team Building	HLD: Analytics Pipeline
37	Coaching	Performance Mgmt.	LLD: Memento
38	Metrics Driven	KPI Review	HLD: Recommendation
39	Security Mindset	Security Review	LLD: Visitor
40	Scalability Mindset	Scalability Planning	HLD: Real-time System
41	Design Thinking	Design Review	LLD: Interpreter
42	Challenge Story	Change Mgmt.	HLD: IoT Platform
43	Why This Company	Org Influence	LLD: Null Object
44	Why This Role	Leadership Mock	HLD: Data Ingestion
45	HR Mock	Executive Mock	LLD + HLD Revision

IMPORTANT NOTES

- Solve problems on paper or in IDE (No Copy-Paste).
- Explain out loud as if you are in an interview.
- Focus on patterns, not just problems.
- Revise previous patterns every weekend.
- Take mocks in the last 10 days seriously.

RESOURCES

- NeetCode 150
- Grokking the Coding Interview (Patterns)
- System Design Primer
- Alex Xu System Design
- Mock Interviews (Pramp, Exponent, Interviewing.io)

DAILY CHECKLIST

- | | |
|--|--|
| ☐ Solved 3 Problems | ☐ Leadership Practice |
| ☐ Explained Solutions | ☐ LLD / HLD Study |
| ☐ Behavioral Practice | ☐ Notes / Revision |

DISCIPLINE TODAY
FREEDOM TOMORROW!



★ ARRAYS & HASHING – CHEAT SHEET (VOLUME 1)

Quikrtr

1. INTERVIEW FLOW

- Clarify**
Constraints, edge cases, input range, duplicates?
- Brute Force**
Write simple solution
- Better**
Optimize Time
- Optimal**
Best approach
- Complexity**
Time & Space
- Code**
Clean & bug free
- Explain**
Walk through

2. PATTERN RECOGNITION

Need uniqueness?	HashSet
Need frequency count?	HashMap
Small fixed alphabet (a-z)?	int[26]
Need grouping?	HashMap<Key, List>
Need Top K frequent?	HashMap + Heap
Need Left & Right products?	Prefix / Suffix
Need sequence (consecutive)?	HashSet
Need matrix validation?	HashSet

3. PROBLEMS COVERED

LC	Problem	Pattern	One-line Memory Trick
217	Contains Duplicate	HashSet	Duplicate $\Rightarrow$!set.add()
242	Valid Anagram	Freq Array	Increment one, decrement other
1	Two Sum	HashMap	Target - current
49	Group Anagrams	HashMap + Freq	26-count key
347	Top K Frequent	HashMap + Heap	Frequency $\rightarrow$ Min Heap
271	Encode & Decode Strings	String Encoding	length#string
238	Product Except Self	Prefix / Suffix	Left $\times$ Right
128	Longest Consecutive Seq.	HashSet	Start only if num-1 absent
36	Valid Sudoku	HashSet	Rows, Columns, Boxes

4. BIGGEST AHA MOMENTS

Group Anagrams

```
key.append(freq).append("#")  
# avoids ambiguity
```

Top K Frequent

HashMap $\rightarrow$ PriorityQueue (min heap)
Keep only K elements

Product Except Self

Brute Force $\rightarrow$ Left[] $\rightarrow$ Right[] $\rightarrow$ Left $\times$ Right
Result[] + rightProduct

Longest Consecutive

Sorting? $\rightarrow$ No $\rightarrow$!contains(num-1) $\rightarrow$ while(contains(num-1))

The if is for avoiding repeated traversals, not for correctness.

Encode & Decode

length#string SRHello
Never use split(). Parse length first.

5. COMPLEXITY QUICK REFERENCE

HashSet contains / add / remove	O(1) avg
HashMap get / put / containsKey	O(1) avg
Array access	O(1)
For loop (n times)	O(n)
Two separate loops	O(2n) = O(n)
Nested loops (n x n)	O(n ²)
Sorting	O(n log n)
PriorityQueue offer / poll	O(log n)

6. BIG-O RULES

Sequential loops

ADD

```
for i in n  
...  
for j in n  
...  
= O(2n) = O(n)
```

Nested loops

MULTIPLY

```
for i in n  
  for j in n  
  ...  
= O(n2)
```

Always drop constants.
Focus on growth.

7. JAVA APIS USED

HashMap

```
put(key, val)  
get(key)  
containsKey(key)  
getOrDefault(key, def)  
computeIfAbsent(key, k -> val)  
keySet() / values()
```

HashSet

```
add(val)  
contains(val)  
remove(val)
```

StringBuilder

```
append(x)  
toString()
```

Arrays

```
sort(arr)  
toString(arr)
```

8. COMMON CONVERSIONS

String $\rightarrow$ char[]	s.toCharArray()
char[] $\rightarrow$ String	new String(arr)
int $\rightarrow$ String	String.valueOf(n) Integer.toString(n)
String $\rightarrow$ int	Integer.parseInt(s)
char $\rightarrow$ int	c - '0'
int $\rightarrow$ char	(char) (n + '0')

9. LAMBDA EXPRESSIONS (QUICK)

Compute If Absent

```
map.computeIfAbsent(key, k -> new ArrayList<>())
```

Comparator (Min Heap)

```
(a, b) -> a - b
```

Comparator (by frequency)

```
(a, b) -> freqMap.get(a) - freqMap.get(b)
```

Sort (custom)

```
arr.sort((a, b) -> b - a)
```

10. WHEN TO USE WHAT?

Use This	When?
HashSet	Check existence / uniqueness
HashMap	Count frequency / key-value
int[26]	Only lowercase letters a-z
Heap (PQ)	Top K / Priority based
Prefix/Suffix	Left/Right contribution problems
HashSet (Seq)	Consecutive sequence problems
HashSet (Matrix)	Row / Col / Box validation

11. TIPS & TRICKS

- Think pattern first, code later.
- Can HashSet replace sorting? Always ask!
- Avoid repeated work (like in Longest Consecutive).
- Use freq array for small fixed ranges (0-25).
- For Top K $\rightarrow$ use Min Heap of size K.
- Edge cases decide AC/TLE. Handle them early.

12. COMMON PITFALLS

- Forgetting to handle empty input
- Not clearing data structure in multiple test cases
- Using split() in Encode/Decode (TLE)
- Wrong comparator in PriorityQueue
- Not checking num-1 before starting sequence (LC 128)

13. MEMORY TRICKS

Set for YES/NO (Exists?)
Map for HOW MANY / VALUE (Count / Store)
Heap for TOP K (Sorted partially)
Left $\times$ Right (Product Except Self)
Start from BEGINNING (num-1 not exists)
Length#String (Encoding rule)
Row, Col, Box (Sudoku rule)

14. COMPLEXITY SUMMARY TABLE

Approach	Time Complexity	Space Complexity	Example Problems
Brute Force (Nested Loop)	O(n ²)	O(1)	Two Sum, Product Except Self
Hashing (Set/Map)	O(n)	O(n)	Contains Duplicate, Two Sum
Sorting	O(n log n)	O(1) / O(n)	Group Anagrams (alt), Longest Consecutive (alt)
Heap (Top K)	O(n log k)	O(k)	Top K Frequent Elements
Prefix / Suffix	O(n)	O(n)	Product Except Self

ARRAYS & HASHING CHEAT SHEET

— ★ Patterns, Tips & Tricks for Coding Interviews ★ —

1 CORE IDEA

- # HashSet → Uniqueness, fast lookups
- HashMap → Frequency, mapping, counts
- Frequency Array → When range is small
- Sorting vs Hashing → Hashing is $O(n)$, Sorting is $O(n \log n)$

2 COMPLEXITY

n = size of input

Operation	HashSet / HashMap	Array / Loop	Sorting
Lookup	$O(1)$ avg	$O(n)$	$O(\log n)$
Insert	$O(1)$ avg	$O(1)$	—
Delete	$O(1)$ avg	$O(n)$	—
Traversal	$O(n)$	$O(n)$	$O(n)$
Space	$O(n)$	$O(1)$	$O(1)$

3 COMMON PATTERNS

- ✓ Unordered data → HashSet / HashMap
- Counting / Frequency → HashMap
- Group by something → HashMap<key, List>
- Check existence → HashSet
- ✗ Avoid duplicates → HashSet
- 💡 Optimize lookup → Replace sorting with Hashing

4 KEY PROBLEMS IN ARRAYS & HASHING

#	Problem	Pattern	Key Idea	Time	Space	Key Tip / Code Snippet
✓ 217	Contains Duplicate	HashSet	If size changes → duplicate exists	$O(n)$	$O(n)$	<code>if (!set.add(x)) return true;</code>
✓ 242	Valid Anagram	HashMap / Array[26]	Count freq of both strings	$O(n)$	$O(1/26)$	<code>count[c]++; count[c]--;</code>
✓ 1	Two Sum	HashMap	Store num → index	$O(n)$	$O(n)$	<code>if (map.containsKey(target-num))</code>
✓ 49	Group Anagrams	HashMap	Sort string or count freq as key	$O(n * k \log k)$	$O(n * k)$	<code>map.computeIfAbsent(key, k-> new ArrayList<>())</code>
✓ 347	Top K Frequent	HashMap + Heap	Count freq + Min Heap (size k)	$O(n \log k)$	$O(n)$	<code>pq.offer(x); if (pq.size() > k) pq.poll();</code>
✓ 271	Encode & Decode	String Builder	Length#string format	$O(n)$	$O(n)$	<code>sb.append(s.length()).append("#").append(s);</code>
✓ 238	Product Except Self	Prefix / Suffix	Left & Right products	$O(n)$	$O(1)$	<code>left[i] * right[i]</code>
✓ 128	Longest Consecutive	HashSet	Start from sequence start only	$O(n)$	$O(n)$	<code>if (!set.contains(x-1)) { while(set.contains(x+1)) x++; }</code>
✓ 36	Valid Sudoku	HashSet	Row, Col, 3x3 box → no duplicates	$O(1)$ (constant)	$O(1)$	Use <code>set.clear()</code> for each row/col/box

5 QUICK TIPS & TRICKS

✓ HashSet Uniqueness

```
Set<Integer> set = new HashSet<>();
for(int x : nums){
    if(!set.add(x)) return true; // duplicate
}
```

Frequency with HashMap

```
Map<Integer, Integer> map = new HashMap<>();
for(int x : nums){
    map.put(x, map.getOrDefault(x, 0) + 1);
}
```

Two Sum Pattern

```
Map<Integer, Integer> map = new HashMap<>();
for(int i=0; i<nums.length, i++){
    int need = target - nums[i];
    if(map.containsKey(need)) return new int[]{map.get(need), i};
    map.put(nums[i], i);
}
```

Group Anagrams (Key by Count)

```
int[] count = new int[26];
for(char c : s.toCharArray()) count[c-'a']++;
StringBuilder key = new StringBuilder();
for(int c : count) key.append(c).append('#');
map.computeIfAbsent(key.toString(), k-> new ArrayList<>()).add(s);
```

Top K Frequent (Min Heap)

```
PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)>>
// [num, freq]
for(int num : map.keySet()){
    pq.offer(new int[]{num, map.get(num)});
    if(pq.size() > k) pq.poll();
}
```

Product Except Self (Without Division)

```
int[] res = new int[n];
int left = 1;
for(int i=0; i<n; i++){ res[i] = left; left *= nums[i]; }
int right = 1;
for(int i=n-1; i>=0; i--){ res[i] *= right; right *= nums[i]; }
```

Longest Consecutive Sequence

```
Set<Integer> set = new HashSet<>(nums);
int longest = 0;
for(int x : set){
    if(!set.contains(x-1)){
        int len = 1;
        while(set.contains(x+1)){ len++; x++; }
        longest = Math.max(longest, len);
    }
}
```

Valid Sudoku (3 Steps)

- Check each row
- Check each column
- Check each 3x3 box

Use `Set<Character> set = new HashSet<>();`
`set.clear()` after each check.

5	3	.	.	7	.	.	.	.
6	.	.	1	9	5	.	.	.
.	9	8	.	.	.	.	6	.
8	.	.	.	6	.	.	.	3
4	.	.	8	.	3	.	.	1
7	.	.	.	2	.	.	.	6
.	6	.	.	.	.	2	8	.
.	.	.	4	1	9	.	.	5
.	.	.	.	8	.	.	7	9

GENERAL REMINDERS

- ★ Think: Can I use HashSet / HashMap?
- ★ Hashing beats sorting when order doesn't matter.
- ★ Always analyze: Time & Space.
- ★ Edge cases make or break solutions.
- ★ Clarity > Cleverness in interviews.



Master the Patterns. Solve with Confidence. Ace the Interviews. 

VOLUME 1: ARRAYS & HASHING

CHEAT SHEET

★ FAANG Interview Edition

🔍 Quick Lookup

⚡ Patterns First, Code Next

1. INTERVIEW FLOW (ALWAYS FOLLOW)

- 1 Clarify**
Ask constraints, edge cases, duplicates, range, complexity.
- 2 Brute Force**
Get it working first.
- 3 Better**
Can we optimize?
- 4 Optimal**
Best approach (Time/Space).
- 5 Complexity**
Time and Space analysis.
- 6 Code**
Clean, bug-free code.
- 7 Explain**
Walkthrough + dry run.

2. PATTERN RECOGNITION

Need	Use	Example Problems
✓ Uniqueness / Duplicate	HashSet	217
📊 Frequency / Count	HashMap / int[26]	242, 347
🔗 Pair / Complement	HashMap	1
👥 Grouping	HashMap<Key, List>	49
📉 Top K Frequent	HashMap + Min Heap	347
🔍 Left & Right Info	Prefix / Suffix	238
🔗 Consecutive Sequence	HashSet	128
🔍 Matrix Validation	HashSet	36
🔍 Encoding / Decoding	Length + Delimiter	271

3. PROBLEMS AT A GLANCE

LC #	Problem	Pattern	Key Idea
217	Contains Duplicate	HashSet	If !set.add(x) ⇒ duplicate
242	Valid Anagram	Frequency Array	Count++ then Count--
1	Two Sum	HashMap	target - num
49	Group Anagrams	HashMap + Freq Sig	26-count key string
347	Top K Frequent	HashMap + Heap	Min Heap of size K
271	Encode & Decode	Length + Delimiter	length#string format
238	Product Except Self	Prefix / Suffix	Left[i] × Right[i]
128	Longest Consecutive	HashSet	Start if num-1 not present
36	Valid Sudoku	HashSet	Rows, Cols, 3x3 boxes

4. KEY PATTERNS & TRICKS (HIGH IMPACT)

4.1 Group Anagrams (LC 49)

```
int[] freq = new int[26];
for(char c : s.toCharArray())
    freq[c - 'a']++;

StringBuilder key = new StringBuilder();
for(int f : freq) key.append(f).append("#");
```

💡 # is important to avoid ambiguity.

4.2 Top K Frequent (LC 347)

```
Map<Integer,Integer> map = new HashMap<>();
// count frequency

PriorityQueue<int[]> pq = new PriorityQueue<>()
    (a,b) -> a[1] - b[1]; // min heap

for(int key : map.keySet()){
    pq.offer(new int[]{key, map.get(key)});
    if(pq.size() > k) pq.poll();
}
```

💡 Keep only K elements in heap.

4.3 Product Except Self (LC 238)

1. Left Product Array
2. Right Product (in one pass)
3. Result[i] = Left[i] * Right

```
nums: [1,2,3,4]
left : [1,1,2,6]
right: [24,12,4,1]
res : [24,12,8,6]
```

💡 No division. O(n) time, O(1) extra space (excluding output).

4.4 Longest Consecutive (LC 128)

```
Set<Integer> set = new HashSet<>();
// add all numbers
int longest = 0;
for(int num : set){
    if(!set.contains(num - 1)){
        int curr = num, count = 1;
        while(set.contains(curr + 1)){
            curr++; count++;
        }
        longest = Math.max(longest, count);
    }
}
```

💡 Start count only if num-1 is absent.

4.5 Encode & Decode (LC 271)

ENCODE:
sb.append(s.length()).append("#").append(s);

DECODE:
Read length until '#', then take that many characters.

Example:
Input : ["Hello","World"]
Encode: 5#Hello5#World
Decode: ["Hello","World"]

💡 Never use split(). Parse length.

5. COMPLEXITY QUICK REFERENCE

Operation	Time Complexity
HashSet add / contains / remove	O(1) avg
HashMap put / get / contains	O(1) avg
Loop (n times)	O(n)
Two separate loops	O(2n) = O(n)
Nested loops (n x m)	O(n*m)
Sorting	O(n log n)
PriorityQueue offer / poll	O(log n)

6. JAVA COLLECTIONS & APIS (MOST USED)

HashSet<Integer> set	HashMap<K,V> map	PriorityQueue<int[]> pq	String / StringBuilder
add(x)	put(k,v)	offer(x)	toCharArray()
contains(x)	get(k)	poll()	String.valueOf(x)
remove(x)	containsKey(k)	peek()	Integer.parseInt(s)
size()	getOrDefault(k, def)	size()	StringBuilder sb
clear()	computeIfAbsent(k, v->newV)	For min heap: (a,b) -> a[1] - b[1]	sb.append(x)
	keySet(), values(), entrySet()		sb.toString()

7. CONVERSIONS (QUICK RECALL)

String → char[]	s.toCharArray()
char[] → String	new String(arr)
int → String	String.valueOf(x) Integer.toString(x)
String → int	Integer.parseInt(s)
char → int	c - '0'
int → char	(char) (x + '0')
List → Array	list.toArray(new T[0])

8. BIG-O RULES (DON'T FORGET)

Sequential loops **ADD**. Nested loops **MULTIPLY**.

Example 1 (Sequential):

```
for i in n
    for j in n
// O(n) + O(n) = O(n)
```

Example 2 (Nested):

```
for i in n
    for j in n
// O(n * n) = O(n^2)
```

One loop = O(n), Two loops sequential = O(n)

9. COMMON MISTAKES (AVOID)

- ✗ Using sort when HashSet can solve in O(n).
- ✗ Forgetting to check num-1 before counting sequence.
- ✗ Using split() in Encode/Decode (causes errors with #).
- ✗ Not handling negative numbers or edge cases.
- ✗ Not analyzing complexity.
- ✗ Over-complicating when a simpler pattern fits.

10. INTERVIEW COMMUNICATION TIPS

- ✓ Think out loud. Explain each step.
- ✓ Start with brute force, then optimize.
- ✓ Give examples and dry run.
- ✓ State assumptions clearly.
- ✓ Write clean, readable code.
- ✓ Summarize time & space at the end.

Golden Line:

"Brute Force → Better → Optimal → Complexity → Code → Explain"

MEMORY MANTRAS



Need **Unique**?
Use **HashSet**

Need **Count**?
Use **HashMap**

Need **Left & Right**?
Use **Prefix / Suffix**

Need **Top K**?
Use **Heap**

Need **Consecutive**?
Use **HashSet**



**PRACTICE. PATTERNS.
REPEAT. CRACK FAANG!**

ARRAYS & HASHING – PATTERNS CHEAT SHEET

Core Patterns, When to Use, Key Ideas & Common Formulas

1 DUPLICATE DETECTION

HashSet

When to use

- Find if any element appears more than once.

Key Idea

- Insert into set, if already present → duplicate.

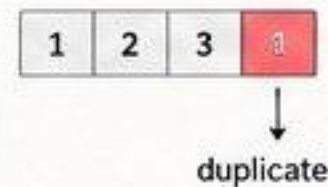
How it Works / Formula / Code

```
Set<Integer> set = new HashSet<>();
for (int x : nums) {
    if (set.contains(x)) return true;
    set.add(x);
}
return false;
```

Example

Input: [1, 2, 3, 1]

Output: true



Time: O(n) | Space: O(n)

2 FREQUENCY COUNTING

HashMap / int[26]

When to use

- Count occurrences of elements.

Key Idea

- Map each element → its frequency.

How it Works / Formula / Code

A) HashMap (any values)

```
Map<Integer, Integer> map = new HashMap<>();
for (int x : nums) {
    map.put(x, map.getOrDefault(x, 0) + 1);
}
```

B) Frequency Array (only for small range e.g. a-z)

```
int[] freq = new int[26];
for (char c : s.toCharArray()) {
    freq[c - 'a']++;
}
```

Example

Input: [1, 2, 2, 3, 3, 3]

Output:
1→1, 2→2, 3→3

Num	Freq
1	1
2	2
3	3

Time: O(n) | Space: O(n)

3 COMPLEMENT / PAIR SEARCH

HashMap

When to use

- Find pair with given sum / target.

Key Idea

- For each x, check if (target - x) exists in map.

How it Works / Formula / Code

```
Map<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (map.containsKey(complement))
        return new int[] {map.get(complement), i};
    map.put(nums[i], i);
}
return new int[2];
```

Example

Input: nums=[2,7,11,15], target=9

Output: [0,1]

Idx	0	1	2	3
	2	7	11	15

↓
9-2=7
found at idx 1

Time: O(n) | Space: O(n)

4 GROUPING / BUCKETING

HashMap<Key, List>

When to use

- Group elements by key (e.g., anagrams, indices, values).

Key Idea

- Map key → list of elements.

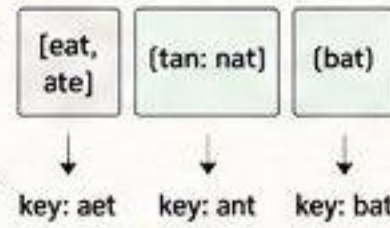
How it Works / Formula / Code

```
Map<String, List<String>> map = new HashMap<>();
for (String s : strs) {
    String key = ...; // e.g. sorted string
    map.computeIfAbsent(key, k -> new ArrayList<>())
        .add(s);
}
```

Example (Group Anagrams)

Input: ["eat","tea","tan","ate","nat","bat"]

Output: [["eat","tea","ate"],["tan","nat"],["bat"]]



Time: O(n * k log k) | Space: O(n * k)
(n = #strings, k = avg length)

5 ENCODING / DECODING STRINGS

StringBuilder

When to use

- Store and retrieve list of strings in single string.

Key Idea

- Prefix each string with its length.

How it Works / Formula / Code

Encode:

```
for (String s : strs)
    sb.append(s.length()).append('#').append(s);
```

Decode:

Read number until '#', then next that many chars is the string.

Example

Input: ["we", "love", "leetcode"]

Encode: "2#we4#love8#leetcode"

Decode: ["we", "love", "leetcode"]

2#we4#love8#leetcode

Time: O(n) | Space: O(n)

6 PREFIX / SUFFIX PATTERNS

Arrays

When to use

- Product except self, prefix sums, suffix sums, etc.

Key Idea

- Precompute left and right contributions.

How it Works / Formula / Code (Product Except Self)

```
left[i] = product of all nums [0 ... i-1]
right[i] = product of all nums [i+1 ... n-1]
ans[i] = left[i] * right[i]
```

```
int n = nums.length;
int[] left = new int[n];
int[] right = new int[n];
int[] ans = new int[n];
left[0] = 1;
for (int i = 1; i < n; i++) left[i] = left[i-1] * nums[i-1];
right[n-1] = 1;
for (int i = n-2; i >= 0; i--) right[i] = right[i+1] * nums[i+1];
for (int i = 0; i < n; i++) ans[i] = left[i] * right[i];
```

Example

nums = [1,2,3,4]
ans = [24,12,8,6]

i	nums	left	right	ans
0	1	1	24	24
1	2	1	12	12
2	3	2	4	8
3	4	6	1	6

Time: O(n) | Space: O(n)

7 LONGEST CONSECUTIVE SEQUENCE

HashSet

When to use

- Find length of longest consecutive elements.

Key Idea

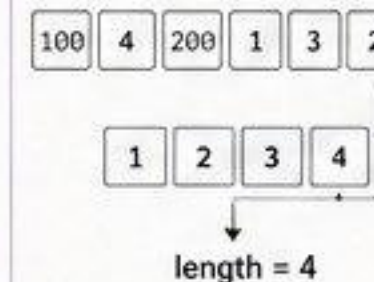
- Only start counting from the first element of a sequence (i.e., num-1 not present).

How it Works / Formula / Code

```
Set<Integer> set = new HashSet<>();
for (int x : nums) set.add(x);
int longest = 0;
for (int x : set) {
    if (!set.contains(x-1)) { // start
        int count = 1, cur = x;
        while (set.contains(cur+1)) {
            cur++; count++;
        }
        longest = Math.max(longest, count);
    }
}
return longest;
```

Example

Input: [100,4,200,1,3,2]
Output: 4
(1,2,3,4)



Time: O(n) | Space: O(n)

8 MATRIX VALIDATION (SUDOKU)

HashSet (Rows, Cols, Boxes)

When to use

- Validate Sudoku board.

- No duplicates in rows, columns and 3x3 boxes.

How it Works

- Check each row using set
- Check each column using set
- Check each 3x3 box using set

3x3 Box Traversal

```
for (int r = 0; r < 9; r += 3)
    for (int c = 0; c < 9; c += 3)
        for (int i = r; i < r+3; i++)
            for (int j = c; j < c+3; j++)
                validate(board[i][j]);
```

Visual – 3x3 Boxes

(0,0)	(0,3)	(0,6)
(3,0)	(3,3)	(3,6)
(6,0)	(6,3)	(6,6)

Time: O(9*9) = O(1) | Space: O(1)

WHEN TO USE WHAT?

✓ HashSet	Fast lookup, duplicates, membership
✓ HashMap	Counts, mapping, complement search, grouping, index mapping
✓ Frequency Array	Small fixed range (e.g., a-z, 0-26)
✗ Sorting	Order matters, two pointers after sort, grouping
✓ Heap / PQ	Top K frequent, K largest/smallest
✓ Arrays	Prefix/Suffix, sliding window, simulation

COMMON HASHING FORMULAS / TRICKS

- Get or Insert: `map.getOrDefault(key, 0) + 1`
- Insert List: `map.computeIfAbsent(key, k -> new ArrayList<>()).add(val)`
- Check Key: `map.containsKey(key)`
- Check Value in Set: `set.contains(val)`
- Remove Key: `map.remove(key)`
- Iterate Entries: `for (Map.Entry<K,V> e : map.entrySet())`
- Convert Values to List: `new ArrayList<>(map.values())`
- Sort Map by Value:

```
map.entrySet().stream()
    .sorted((a,b) -> b.getValue() - a.getValue())
    .collect(Collectors.toList());
```

CHAR / ASCII TRICKS

- char to index: `c - 'a' // 0 to 25`
- index to char: `(char)('a' + i)`
- char to int digit: `c - '0'`
- int to char digit: `(char)('0' + x)`
- Uppercase: `c - 'A' 0 to 25`

FREQUENCY ARRAY TEMPLATE (a-z)

```
int[] freq = new int[26];
for (char c : s.toCharArray())
    freq[c - 'a']++;
```

TIME COMPLEXITY QUICK GUIDE (Average Case)

Operation	HashSet	HashMap
Add / Put	O(1)	O(1)
Remove	O(1)	O(1)
Contains / Get	O(1)	O(1)
Iterate All	O(n)	O(n)

IMPORTANT NOTES

- Hashing gives O(1) average lookup but O(n) worst case (rare with good hash function).
- Choose the right pattern – it saves time and code.
- Think: Can I transform this into a lookup problem?

★ Golden Rule: Understand the Pattern → Choose the Right Data Structure → Code Clean → Analyze Complexity → Optimize if Needed

ARRAYS & HASHING CHEAT SHEET

PROBLEM	KEY IDEA	APPROACH / FORMULA / PATTERN	TIME COMPLEXITY	SPACE COMPLEXITY	KEY TAKEAWAYS / PATTERNS
1. 217 Contains Duplicate (Easy)	Use HashSet to track seen elements.	<pre>Set<Integer> set = new HashSet<>(); for (int x : nums) { if (set.contains(x)) return true; set.add(x); } return false;</pre>	O(n)	O(n)	- HashSet for fast lookup - Detect duplicates
2. 242 Valid Anagram (Easy)	Two strings are anagrams if both have same character frequency.	<pre>int[26] count = new int[26]; for (char c : s.toCharArray()) count[c-'a']++; for (char c : t.toCharArray()) count[c-'a']--; return all count[i] == 0;</pre>	O(n) (n = length of string)	O(1) (26 letters)	- Frequency counting - Array of size 26 - Increment / decrement
3. 1 Two Sum (Easy)	Store number → index in a HashMap. For each num, check if target - num exists.	<pre>Map<Integer,Integer> map = new HashMap<>(); for (int i = 0; i < nums.length; i++) { int comp = target - nums[i]; if (map.containsKey(comp)) return new int[]{map.get(comp), i}; map.put(nums[i], i); }</pre>	O(n) (n = length of array)	O(n)	- HashMap for complement - Store index, not value
4. 49 Group Anagrams (Medium)	Sort each string to create a key. Anagrams have same sorted key.	<pre>Map<String, List<String>> map = new HashMap<>(); for (String s : strs) { char[] arr = s.toCharArray(); Arrays.sort(arr); String key = new String(arr); map.computeIfAbsent(key, k -> new ArrayList<>()).add(s); } return new ArrayList<>(map.values());</pre>	O(n * k log k) (n = #strings, k = avg length)	O(n * k)	- Sorting as key - HashMap<String, List<String>>
5. 347 Top K Frequent Elements (Medium)	Count frequency using HashMap, then use bucket sort (freq → list of numbers).	<pre>1. Count freq: Map<Integer, Integer> freq = new HashMap<>(); 2. Bucket sort: List<Integer>[] bucket = new List[nums.length + 1]; for (int num : freq.keySet()) { int f = freq.get(num); if (bucket[f] == null) bucket[f] = new ArrayList<>(); bucket[f].add(num); } 3. Traverse from high freq to low, collect elements.</pre>	O(n) (n = array length)	O(n)	- Frequency Map + Bucket Sort - Buckets sized by frequency - Read from high to low
6. 271 Encode and Decode Strings (Medium)	Use length-prefix encoding: length#string to uniquely separate strings.	<pre>Encode: for (String s : strs) sb.append(s.length()).append("#").append(s); Decode: read number until '#', then next that many chars is the string.</pre>	Encode: O(n) Decode: O(n) (n = total length of all strings)	O(n)	- Length-prefix avoids ambiguity - Unique & decodable
7. 238 Product of Array Except Self (Medium)	Use prefix & suffix products. Avoid division.	<pre>int n = nums.length; int[] left = new int[n], right = new int[n], ans = new int[n]; left[0] = 1; for (int i=1; i<n; i++) left[i] = left[i-1] * nums[i-1]; right[n-1] = 1; for (int i=n-2; i>=0; i--) right[i] = right[i+1] * nums[i+1]; for (int i=0; i<n; i++) ans[i] = left[i] * right[i];</pre>	O(n)	O(n) (for output array)	- Prefix products & suffix products - No division - Left[i] * Right[i]
8. 128 Longest Consecutive Sequence (Hard)	Use HashSet. Start counting only from the beginning of a sequence (num-1 not present).	<pre>Set<Integer> set = new HashSet<>(); for (int num : nums) set.add(num); int longest = 0; for (int num : set) { if (!set.contains(num - 1)) { // start of sequence int count = 1, cur = num; while (set.contains(cur + 1)) { count++; cur++; } longest = Math.max(longest, count); } }</pre>	O(n)	O(n)	- HashSet for O(1) lookup - Start from sequence beginning (num-1 not present) - Each number visited once
9. 36 Valid Sudoku (Medium)	Check rows, columns and 3x3 sub-boxes using HashSet.	<pre>1. For each row → use Set<Character> 2. For each column → use Set<Character> 3. For each 3x3 box → use Set<Character> 9. Box loops: for (int r=0; r<9; r+=3; r++) for (int cc=0; cc<9; cc+=3) for (int i=r; i<r+3; i++) for (int j=cc; j<cc+3; j++)</pre>	O(1) (Board size is fixed 9x9)	O(1) (Fixed size sets)	- Use HashSet for rows, columns and 3x3 boxes - Box traversal: jump by 3

COMMON HASHING DATA STRUCTURES	FREQUENCY COUNT PATTERN	BUCKET SORT (BY FREQUENCY)	3x3 BOX TRAVERSAL (SUDOKU)	COMPLEXITY REMINDER
- HashSet<T> → Uniqueness, Fast lookup - HashMap<K,V> → Key-Value, Frequency count - TreeMap<K,V> → Sorted keys - LinkedHashMap → Maintains insertion order	<pre>Map<Integer, Integer> map = new HashMap<>(); for (int num : nums) { map.put(num, map.getOrDefault(num, 0) + 1); }</pre>	<pre>List<Integer>[] bucket = new List[nums.length + 1]; for (int num : map.keySet()) { int f = map.get(num); if (bucket[f] == null) bucket[f] = new ArrayList<>(); bucket[f].add(num); }</pre>	<pre>for (int r = 0; r < 9; r += 3) { for (int c = 0; c < 9; c += 3) { for (int i = r; i < r + 3; i++) { for (int j = c; j < c + 3; j++) { // process board[i][j] } } } }</pre>	- HashSet / HashMap Operations (avg): add, remove, contains, get, put → O(1) - Sorting → O(n log n) - Traversal → O(n)

★ **REMEMBER:** Use the right tool for the right problem. Hashing improves lookup from O(n) to O(1) on average.

ARRAYS & HASHING – PATTERNS CHEAT SHEET (IMAGE 1)

Master these patterns. They solve **80%** of Array + Hashing problems in interviews.

1. DUPLICATE DETECTION

Use Case: Check if any element appears more than once.

Data Structure: HashSet

Key Idea:

Add elements to a set.
If already present → duplicate found.

```
Set<Integer> set = new HashSet<>();
for (int x : nums) {
    if (!set.add(x)) return true; // already exists
}
return false;
```

Problems

- 217. Contains Duplicate
- 219. Contains Duplicate II
- 220. Contains Duplicate III

Time Complexity: O(n)

Space Complexity: O(n)

2. FREQUENCY COUNTING

Use Case: Count frequency of each element / character.

Data Structure: HashMap / int[26] (for lowercase letters)

Key Idea:

Count occurrences and store.
Helps in anagrams, top K freq, majority element etc.

HashMap approach:

```
Map<Integer, Integer> map = new HashMap<>();
for (int x : nums) {
    map.put(x, map.getOrDefault(x, 0) + 1);
}
```

Array approach (chars 'a'-'z'):

```
int[] freq = new int[26];
for (char c : s.toCharArray()) freq[c - 'a']++;
```

Problems

- 242. Valid Anagram
- 383. Ransom Note
- 49. Group Anagrams
- 347. Top K Frequent
- 169. Majority Element

Time Complexity: O(n)

Space Complexity: O(n)

3. COMPLEMENT / PAIR SEARCH

Use Case: Find if a pair exists with given sum / target.

Data Structure: HashMap (value → index)

Key Idea:

For each element x, check if (target - x) exists.
Store seen elements with their index.

```
Map<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (map.containsKey(complement))
        return new int[]{map.get(complement), i};
    map.put(nums[i], i);
}
return new int[0];
```

Problems

- 1. Two Sum
- 167. Two Sum II
- 560. Subarray Sum = K
- 454. 4Sum II

Time Complexity: O(n)

Space Complexity: O(n)

4. GROUPING / BUCKETING

Use Case: Group elements by a key (anagram, index diff, etc).

Data Structure: HashMap<Key, List/Count>

Key Idea:

Use key as group identifier and store list of elements.
Very useful in anagram problems.

```
Map<String, List<String>> map = new HashMap<>();
for (String s : strs) {
    char[] arr = s.toCharArray();
    Arrays.sort(arr);
    String key = new String(arr);
    map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
}
return new ArrayList<>(map.values());
```

Problems

- 49. Group Anagrams
- 438. Find All Anagrams in a String
- 128. Longest Consecutive Sequence (variant)

Time Complexity: O(n * k log k)
(k = avg length of string)

Space Complexity: O(n * k)

5. ENCODING / SERIALIZATION

Use Case: Encode list of strings / decode back.

Pattern: length#string

Key Idea:

Prefix each string with its length and a delimiter.
Avoid ambiguity ("ab#c" vs "a#bc").

```
// Encode
StringBuilder sb = new StringBuilder();
for (String s : strs) sb.append(s.length()).append('#').append(s);

// Decode
List<String> res = new ArrayList<>();
for (int i = 0; i < s.length(); ) {
    int j = i;
    while (s.charAt(j) != '#') j++;
    int len = Integer.parseInt(s.substring(i, j));
    i = j + 1;
    res.add(s.substring(i, i + len));
    i += len;
}
return res;
```

Problems

- 271. Encode and Decode Strings

Time Complexity: O(n)

Space Complexity: O(n)

6. PREFIX / SUFFIX TECHNIQUE

Use Case: Product except self, prefix sums, suffix sums.

Key Idea:

Precompute left (prefix) and right (suffix) values.
Combine to get result in O(n) without extra structures.

```
int n = nums.length;
int[] left = new int[n], right = new int[n], ans = new int[n];
left[0] = 1;
for (int i = 1; i < n; i++) left[i] = left[i-1] * nums[i-1];
right[n-1] = 1;
for (int i = n-2; i >= 0; i--) right[i] = right[i+1] * nums[i+1];
for (int i = 0; i < n; i++) ans[i] = left[i] * right[i];
```

Problems

- 238. Product of Array Except Self
- 724. Find Pivot Index

Time Complexity: O(n)

Space Complexity: O(1)
(excluding output array)

7. LONGEST CONSECUTIVE SEQUENCE

Use Case: Find length of longest sequence of consecutive integers.

Data Structure: HashSet

Key Idea:

Store all numbers in a set.
Start counting only from the beginning of a sequence (i.e., num-1 not present) to avoid duplicates.

```
Set<Integer> set = new HashSet<>();
for (int x : nums) set.add(x);
int longest = 0;
for (int x : set) {
    if (!set.contains(x - 1)) { // start of sequence
        int cur = x, count = 1;
        while (set.contains(cur + 1)) { cur++; count++; }
        longest = Math.max(longest, count);
    }
}
return longest;
```

Problems

- 128. Longest Consecutive Sequence

Time Complexity: O(n)

Space Complexity: O(n)

8. MATRIX VALIDATION (USING HASHING)

Use Case: Validate rows, columns, and sub-boxes (e.g., Sudoku).

Key Idea:

Use HashSet for each row, column and 3x3 box to check duplicates.

Rows:

For each row i, use a set.
Check all 9 elements.

Columns:

For each col j, use a set.
Check all 9 elements.

3x3 Boxes:

For top-left corners (0,0), (0,3), (0,6), (3,0), (3,3), (3,6), (6,0), (6,3), (6,6) check 3x3 elements.

Problems

- 36. Valid Sudoku

5	3	.	.	2	.	.	.	1
2	2	1	.	7	.	7	2	.
.	1	1	3	.	2	.	.	1
1	1	.	.	3	.	.	.	1
3	1	.	.	1	.	.	3	.
2	1	1	.	3	2	2	3	.
1	3	3	.	3	.	1	1	3
1	.	2	1	3	1	.	.	.
1	.	.	3	3	1	.	.	.

Time Complexity: O(1) → fixed 9x9 board (General: O(n²))

Space Complexity: O(1)

WHEN TO USE WHAT?

Scenario	Best Choice
Check existence / duplicates	HashSet
Count frequency / occurrence	HashMap or Frequency Array
Find pair / complement	HashMap
Group by key	HashMap<Key, List>
Top K frequent elements	HashMap + Heap (PriorityQueue)
Range / consecutive sequence	HashSet (sequence start)
Product except self / prefix tasks	Prefix & Suffix Arrays
Matrix / Sudoku validation	HashSet per row/col/box

TIPS & REMINDERS

- Always think: Can I get O(1) average lookup? → Use HashSet / HashMap
- Read constraints carefully (size, value range, duplicates allowed?).
- Choose the simplest DS that fits the problem.
- Pay attention to space vs time trade-off.

COMMON HASHMAP METHODS (JAVA)

map.put(key, val) → insert/update
map.get(key) → get value (null if not present)
map.getOrDefault(key, def) → get with default
map.containsKey(key) → check key exists
map.computeIfAbsent(key, k -> new ArrayList<>()).add(val);

COMPLEXITY REFERENCE

Operation	HashMap / HashSet	Array	Sorting
Lookup	O(1) avg	O(1)	O(log n)
Insert	O(1) avg	O(1) amortized	N/A
Delete	O(1) avg	O(n)	N/A
Iteration	O(n)	O(n)	O(n log n)

SPACE RULE OF THUMB

If n is the size of input:
• HashSet / HashMap → O(n)
• Frequency Array (26) → O(1)
• Prefix / Suffix → O(n) or O(1)
(extra space excluding output)

🏆 **REMEMBER: Patterns > Problems. Master the pattern, solve any problem!**

ARRAYS & HASHING – PROBLEMS, COMPLEXITY & TEMPLATES CHEAT SHEET (IMAGE 3)

🧠 Problems → Approach / Key Idea → Time Complexity

A. PROBLEM WISE QUICK REFERENCE

#	PROBLEM	KEY IDEA / APPROACH	TIME	SPACE	IMPORTANT NOTES / PATTERN
1	217. Contains Duplicate	HashSet – add while iterating, if already exists → duplicate.	O(n)	O(n)	Typical use: seen/visited tracking
2	242. Valid Anagram	Frequency count (int[26]) or HashMap<Character, Integer>	O(n)	O(1)	Count & compare all frequencies
3	1. Two Sum	HashMap (value → index). Check complement (target - num).	O(n)	O(n)	Store index, not just value
4	49. Group Anagrams	Key = sorted string or 26-char frequency string. HashMap<Key, List<String>>	O(n * k log k)	O(n * k)	k = avg length of string
5	347. Top K Frequent Elements	HashMap for freq + Bucket Sort (freq → list) or Min-Heap (size k).	O(n) Heap: O(n log k)	O(n)	Bucket sort is O(n) time
6	271. Encode and Decode Strings	Encode: len#string Decode: read length, then string	O(n)	O(n)	# as delimiter avoids ambiguity
7	238. Product of Array Except Self	Prefix products (left) + Suffix products (right) No division allowed.	O(n)	O(n)	res[i] = left[i-1] * right[i+1]
8	128. Longest Consecutive Sequence	HashSet – store all numbers. Start count only if !contains(num-1)	O(n)	O(n)	Avoid duplicates by this trick
9	36. Valid Sudoku	HashSet for Rows, Columns and 3x3 boxes (fixed 9x9 board)	O(1)	O(1)	3 loops → rows, cols, boxes
10	Bonus Pattern: Character Count (lowercase)	Use int[26], index = char - 'a'	O(n)	O(1)	Fastest for lowercase letters

B. TEMPLATES YOU SHOULD REMEMBER

1. HASHSET – DUPLICATE CHECK

```
Set<Integer> set = new HashSet<>();
for (int x : nums) {
    if (!set.add(x)) return true; // already exists
}
return false;
```

2. HASHMAP – FREQUENCY COUNT

```
Map<Integer, Integer> map = new HashMap<>();
for (int x : nums) {
    map.put(x, map.getOrDefault(x, 0) + 1);
}
```

3. TWO SUM (HASHMAP)

```
Map<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
    int need = target - nums[i];
    if (map.containsKey(need)) return new int[]{
        map.get(need), i};
    map.put(nums[i], i);
}
return new int[0];
```

4. GROUP ANAGRAMS (SORTED STRING KEY)

```
Map<String, List<String>> map = new HashMap<>();
for (String s : strs) {
    char[] arr = s.toCharArray();
    Arrays.sort(arr);
    String key = new String(arr);
    map.computeIfAbsent(key, k -> new
    ArrayList<>()).add(s);
}
return new ArrayList<>(map.values());
```

5. TOP K FREQUENT (BUCKET SORT)

```
Map<Integer, Integer> freq = new HashMap<>();
for (int x : nums) freq.put(x, freq.getOrDefault(x, 0) + 1);
List<Integer>[] bucket = new List[nums.length + 1];
for (int num : freq.keySet()) {
    int f = freq.get(num);
    if (bucket[f] == null) bucket[f] = new ArrayList<>();
    bucket[f].add(num);
}
List<Integer> res = new ArrayList<>();
for (int f = bucket.length - 1; f >= 0 && res.size() < k; f--) {
    if (bucket[f] != null) res.addAll(bucket[f]);
}
return res.subList(0, k);
```

6. LONGEST CONSECUTIVE SEQUENCE

```
Set<Integer> set = new HashSet<>(nums);
int longest = 0;
for (int num : set) {
    if (!set.contains(num - 1)) { // start of sequence
        int cur = num, len = 1;
        while (set.contains(cur + 1)) { cur++; len++; }
        longest = Math.max(longest, len);
    }
}
return longest;
```

7. VALID SUDOKU (3 CHECKS)

Rows	Columns	3x3 Boxes
<pre>for (int i = 0; i < 9; i++) { Set<Character> seen = new HashSet<>(); for (int j = 0; j < 9; j++) { char c = board[i][j]; if (c != '.' && !seen.add(c)) return false; } }</pre>	<pre>for (int j = 0; j < 9; j++) { Set<Character> seen = new HashSet<>(); for (int i = 0; i < 9; i++) { char c = board[i][j]; if (c != '.' && !seen.add(c)) return false; } }</pre>	<pre>for (int br = 0; br < 9; br += 3) { for (int bc = 0; bc < 9; bc += 3) { Set<Character> seen = new HashSet<>(); for (int i = br; i < br + 3; i++) { for (int j = bc; j < bc + 3; j++) { char c = board[i][j]; if (c != '.' && !seen.add(c)) return false; } } } }</pre>

C. COMMON PATTERNS SUMMARY

PATTERN	HOW TO THINK
Seen / Visited	Use HashSet
Counting / Frequency	HashMap or int[26]
Complement Search	HashMap (value → index)
Grouping	HashMap<Key, List>
Top K / Min/Max K	Heap or Bucket Sort
Consecutive Sequence	HashSet + check (num-1)
Matrix Validation	Separate set for row/col/box
Prefix / Suffix	Two passes (left → right, right → left)
Encoding	length#string
Avoid Division	Prefix & suffix products
Alphabet Count	int[26] with (c - 'a')

D. COMPLEXITY QUICK GUIDE

OPERATION	TIME COMPLEXITY
HashSet add/contains/remove	O(1) avg
HashMap get/put/containsKey	O(1) avg
Array access	O(1)
Traverse Array	O(n)
Sort (Arrays.sort)	O(n log n)
Building Map/Set from n items	O(n)
Heap push / pop	O(log n)
Bucket Sort (by frequency)	O(n)
Sudoku (fixed 9x9)	O(1)

E. HASHMAP / HASHSET REMINDERS

HashMap Notes

- map.put(key, value)
- map.get(key)
- map.getOrDefault(key, defaultValue)
- map.containsKey(key)
- map.containsValue(value)
- map.remove(key)
- map.size(), map.isEmpty()
- map.keySet(), map.values(), map.entrySet()

HashSet Notes

- set.add(x) → returns false if already exists
- set.contains(x)
- set.remove(x)
- set.size(), set.isEmpty()
- set.clear()
- set → new ArrayList<>(set) to convert

F. FREQUENTLY USED STRUCTURES

Structure	When to Use
HashSet	Check existence, remove duplicates
HashMap	Count frequency, store mapping
int[26]	Lowercase character frequency
ArrayList / List	Dynamic list, grouping results
PriorityQueue	Top K, Min/Max problems
Queue / Deque	BFS / Sliding Window / Monotonic Queue
Stack	Monotonic Stack, Next Greater, Valid Parentheses

G. TIME COMPLEXITY OF COMMON ALGORITHMS

Algorithm / Approach	Time Complexity	Best Case / Notes
Brute Force (single loop)	O(n)	—
Brute Force (nested loops)	O(n^2)	—
Hashing (build map/set)	O(1)	O(1) avg per op
Sorting	O(n log n)	Timsort in Java
Two Pointers (sorted)	O(n)	After sorting if needed
Prefix / Suffix	O(n)	Two linear passes
Sliding Window	O(n)	Each element visited at most twice

H. MEMORY USAGE GUIDE

Structure	Space Usage
HashSet (n elements)	O(n)
HashMap (n pairs)	O(n)
int[26]	O(1)
Array / List (n)	O(n)
Stack / Queue (n)	O(n)
PriorityQueue (n)	O(n)

ARRAYS & HASHING – JAVA CHEAT SHEET (IMAGE 2)

Essential Java APIs, Syntax & Utilities you use in Arrays + Hashing problems

1. DATA STRUCTURES – QUICK REFERENCE

HashSet<E> (Unique Elements)

```
Set<Character> set = new HashSet<>();
set.add('a');           // true if added
set.contains('a');      // O(1) avg
set.remove('a');
set.size();
set.isEmpty();
set.clear();
Time Complexity: add/contains/remove -> O(1) avg
```

HashMap<K, V> (Key → Value)

```
Map<String, Integer> map = new HashMap<>();
map.put("a", 1);         // insert/update
map.get("a");            // returns value or null
map.containsKey("a");    // default if not present
map.containsValue(1);
map.remove("a");
map.size(); map.isEmpty(); map.clear();
Time Complexity: put/get/containsKey -> O(1) avg
```

LinkedHashMap<K, V> (Maintains Insertion Order)

```
Map<Integer, Integer> lhm = new LinkedHashMap<>();
lhm.put(1, 10); lhm.put(2, 20);
// Iterates in insertion order
```

TreeMap<K, V> (Sorted by Key)

```
Map<Integer, String> tm = new TreeMap<>();
tm.put(3, "c"); tm.put(1, "a"); tm.put(2, "b");
// Keys iterated in sorted order: 1, 2, 3
Time Complexity: O(log n)
```

PriorityQueue<E> (Min-Heap by default)

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(5); pq.add(1); pq.add(3);
pq.peek(); // 1 (min)
pq.poll(); // 1
pq.size(); pq.isEmpty();
// For Max-Heap:
PriorityQueue<Integer> pqMax = new PriorityQueue<>(
    Collections.reverseOrder());
Time Complexity: add/poll/peek -> O(log n)
```

2. COMMON JAVA CONVERSIONS

String ↔ char[]

```
String s = "abc";
char[] arr = s.toCharArray();
String s2 = new String(arr);
```

String ↔ int

```
String s = "123";
int x = Integer.parseInt(s);
String s2 = String.valueOf(x);
// or
String s3 = x + "";
```

List ↔ Array

```
List<Integer> list = Arrays.asList(1,2,3);
// Fixed-size list backed by array
Integer[] arr = list.toArray(new Integer[0]);
int[] arr2 = list.stream().mapToInt(i -> i)
    .toArray();
```

Array ↔ List (Mutable)

```
Integer[] arr = {1,2,3};
List<Integer> list = new ArrayList<>(
    Arrays.asList(arr));
```

Set ↔ List / Array

```
Set<Integer> set = new HashSet<>();
set.add(1); set.add(2);
List<Integer> list = new ArrayList<>(set);
Integer[] arr = set.toArray(new Integer[0]);
```

Map Values / Keys

```
Collection<Integer> values = map.values();
Set<String> keys = map.keySet();
List<Integer> valuesList = new ArrayList<>(
    map.values());
```

StringBuilder (for Encoding / Building Strings)

```
StringBuilder sb = new StringBuilder();
sb.append('a'); sb.append("bc"); sb.append(123);
sb.length(); sb.charAt(0); sb.toString();
sb.setLength(0); // clear
```

Math Utilities

```
Math.max(a, b); Math.min(a, b);
Math.abs(x); Math.pow(a, b);
Math.floor(x); Math.ceil(x);
```

4. FREQUENCY ARRAY (For Lowercase Letters)

```
int[] freq = new int[26];
for (char c : s.toCharArray()) {
    freq[c - 'a']++;
}
// index = char - 'a' (0 to 25)
// Works for 'a' to 'z' only
```

5. BUCKET / GROUPING PATTERN

```
Map<String, List<String>> map = new HashMap<>();
for (String s : strs) {
    char[] arr = s.toCharArray();
    Arrays.sort(arr);
    String key = new String(arr);
    map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
}
List<List<String>> res = new ArrayList<>(map.values());
```

3. USEFUL JAVA PATTERNS & METHODS

computeIfAbsent() – Very Important!

```
Map<Character, List<Integer>> map = new HashMap<>();
map.computeIfAbsent('a', k -> new ArrayList<>()).add(1);
// If key 'a' not present, creates new ArrayList<>
// then adds 1. Else, directly adds to existing list.
```

getOrDefault()

```
int count = map.getOrDefault(key, 0);
// returns value if key exists, else default (0)
```

merge() – Another Powerful One

```
map.merge(key, 1, Integer::sum);
// If key absent -> put(key, 1)
// If present -> oldVal + 1
```

forEach() with Lambda

```
map.forEach((key, val) -> {
    System.out.println(key + " -> " + val);
});
```

Stream APIs (Often Used)

```
List<Integer> list = new ArrayList<>(map.values());
list.stream().sorted().toList();
list.stream().sorted(Comparator.reverseOrder()).toList();
list.stream().limit(k).toList();
list.stream().mapToInt(i -> i).toArray();
Arrays.stream(arr).boxed().collect(Collectors.toList());
```

Arrays Utility

```
Arrays.sort(arr);
Arrays.sort(arr, Comparator.reverseOrder());
Arrays.fill(arr, 0);
Arrays.copyOf(arr, newLength);
Arrays.equals(arr1, arr2);
```

Queue / Deque

```
Queue<Integer> q = new LinkedList<>();
q.offer(1); q.poll(); q.peek();
Deque<Integer> dq = new ArrayDeque<>();
dq.offerFirst(1); dq.offerLast(2);
dq.pollFirst(); dq.pollLast(); dq.peekFirst(); dq.peekLast();
```

Stack (Legacy) or Deque (Preferred)

```
Stack<Integer> st = new Stack<>();
st.push(1); st.pop(); st.peek(); st.isEmpty();
Deque<Integer> st2 = new ArrayDeque<>(); // better
st2.push(1); st2.pop(); st2.peek(); st2.isEmpty();
```

8. QUICK SYNTAX REMINDER (One-Liners You Use Often)

Iterate HashMap

```
for (Map.Entry<K, V> e : map.entrySet()) {
    K key = e.getKey();
    V val = e.getValue();
}
```

Iterate HashSet

```
for (E e : set) {
    // use e
}
```

Create List

```
List<Integer> list = new ArrayList<>();
List<Integer> list2 = Arrays.asList(1,2,3);
List<Integer> list3 = new ArrayList<>(list2);
```

Convert List to Array

```
Integer[] arr = list.toArray(new Integer[0]);
int[] arr2 = list.stream().mapToInt(i -> i)
    .toArray();
```

Sort List

```
Collections.sort(list);
list.sort(Comparator.reverseOrder());
```

Reverse String

```
new StringBuilder(s)
    .reverse().toString();
```

• MOST USED IN INTERVIEWS

- HashSet
- HashMap + computeIfAbsent
- getOrDefault / merge
- PriorityQueue
- Arrays.sort(), toArray(), asList()
- StringBuilder



REMEMBER



Pattern Recognition > Code



Know your Tools



Write Clean & Simple Code



Analyze Complexity



Practice Consistently